

Mathematical guide to LDA-PIBT

LDA-PIBT — Lifelong Aisle-Managed Priority Inheritance with Backtracking

AisleFlow · generated from docs/mathematical-guide.md by tools/build_docs.py

A self-contained reference to every algorithm, formula and symbol in this codebase. Where `docs/implementation-notes.md` maps spec sections to code, this document explains the math itself: what each formula computes, why it has the shape it does, and what every symbol means. Section numbers below are file/module based, not spec-section based (see `implementation-notes.md` for the spec cross-reference).

Notation used throughout, matching the codebase's own docstrings:

- `i`, `j` — robot indices. `t` — the current timestep (an integer; the simulator advances it by exactly 1 per `Simulator.step()` call).
- `v`, `u`, `x` — grid vertices, i.e. `(row, col)` pairs. `x_i(t)` is robot `i`'s position at time `t`.
- `g`, `w` — a goal/waypoint vertex.
- Greek letters (`α`, `β`, `γ`, `λ`, `μ`, `ν`, `ξ`, `ω`, `τ`, `η`) are tunable weights, all fields of `config.Params`; their default numeric values are given inline and collected in §15.
- `1[condition]` is the indicator function: `1` if true, `0` otherwise.
- `INF = float("inf")`, used as the "unreachable" distance sentinel throughout (`types.INF`).

How to read this

This document is a reference, not a tutorial: it is organised by module, and every section states its formulas exactly as the code computes them. That makes it thorough and slow to read front-to-back, so three routes through it are worth naming.

To get oriented (about 20 minutes). Read §0 for the two-layer architecture, §11 for the sixteen-step loop that runs every timestep, and keep §16 — the glossary of every symbol — open beside whatever else you read. Those three answer "what is this system doing?"

To understand the contribution (about an hour). Add §4 (the candidate score, which is where every high-level preference is finally cashed out), §6 (aisle direction, the actual novelty), and §10 (deadlock detection and recovery, which is what makes §6 survivable). Each of those carries a worked example computed from a live run — real robots, real numbers, on `warehouse_medium`.

As a reference. Everything else. §15 is the complete parameter list with defaults and the ablation ladder; `docs/implementation-notes.md` maps spec sections to functions; the README's Results section reports which of these mechanisms actually earn their cost.

Five **worked examples** are marked in the text. Every number in them is computed by `tools/worked_examples.py`, which imports the same modules the simulator does and runs them on a fixed scenario; `tests/test_worked_examples.py` fails if this document and the code ever disagree. They are the fastest way to see what a formula really does.

Contents

How to read this

0. The two-layer architecture

1. Graph and warehouse model (`graph.py` , `warehouse.py`)

- 1.1 Grid graph
- 1.2 BFS distance maps
- 1.3 Structural features
- 1.4 Aisle segmentation

2. Robots and tasks (`robot.py` , `task.py`)

- 2.1 Robot state
- 2.2 Task urgency and waiting
- 2.3 Arrival processes (`TaskGenerator`)

3. Priority function (`priority.py`)

- 3.1 Task-class priority
- 3.2 Fairness horizon
- 3.3 Tie-breaking and ordering

4. Candidate scoring (`scoring.py`)

- 4.1 Proximity mode
- 4.2 Smooth direction weight β_i
- 4.3 Aisle-continuity weight γ_i
- 4.4 Turning cost
- 4.5 Progress
- 4.6 The full candidate score $S_i(v)$
- 4.7 Preferred route direction and hysteresis

5. Congestion model (`congestion.py`)

- 5.1 Occupancy index
- 5.2 Downstream lookahead
- 5.3 Congestion mixture
- 5.4 Route-level congestion (for task assignment)

6. Aisle management (`aisle_manager.py`)

- 6.1 Requested direction
- 6.2 Per-robot directional demand
- 6.3 Aggregate demand and the decision rule
- 6.4 Aisle state machine
 - 6.4.1 Coordinated direction assignment
- 6.5 Reservations

- 6.6 Movement legality (spec 27, `violates_aisle_direction`)

7. PIBT core (`pibt.py`)

- 7.1 Candidate set
- 7.2 Hard rejection rules (applied in this exact order)
- 7.3 Conflict checks (`validate.py` companions, spec 26)
- 7.4 The PIBT recursion (spec 25)
- 7.5 Complexity

8. Routing (`routing.py`)

9. Task assignment (`assignment.py`)

- 9.1 Directional delay
- 9.2 Blocking estimate
- 9.3 Full assignment cost $J(i, \tau)$
- 9.4 Greedy matching

10. Deadlock detection and recovery (`deadlock.py`)

- 10.1 Three distinct stall signals
- 10.2 Progress signal
- 10.3 Repeated-configuration and dependency-graph detection
- 10.4 Seven-level progressive recovery

11. The lifelong simulation loop (`simulator.py`)

12. Evaluation metrics (`metrics.py`)

- 12.1 Percentile
- 12.2 Jain's fairness index
- 12.3 Aggregate report fields

13. Statistics (`stats.py`)

- 13.1 Percentile bootstrap confidence interval
- 13.2 Permutation test

14. Baseline algorithms (`baselines/`)

- 14.1 Time-expanded A* (`space_time_search.py`)
- 14.2 Token Passing (Ma, Kumar, Koenig & Ayanian, 2017)
- 14.3 RHCR — Rolling-Horizon Collision Resolution (Li, Tinka, Kiesel, Durham, Kumar & Koenig, 2021)

15. Parameter reference (`config.Params`)

- 15.1 The ablation ladder and factorial designs

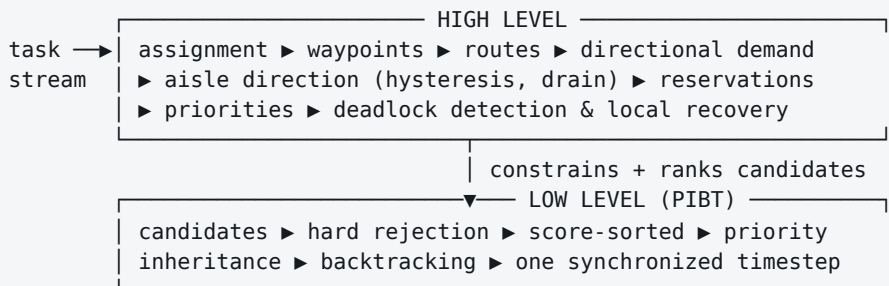
16. Glossary of symbols

Cross-reference

0. The two-layer architecture

In one sentence. Every mechanism in this document sits in one of two layers: an upper layer that decides what each robot *wants*, and a lower layer that decides what is *allowed*. Only the lower layer can prevent a collision, and nothing in the upper layer is permitted to remove a move the lower layer would have considered.

LDA-PIBT extends **Priority Inheritance with Backtracking** (PIBT; Okumura, Machida, Défago, Tamura, arXiv:1901.11282) — a single-timestep, decentralized multi-agent collision-avoidance algorithm — with a high-level layer that decides *what* each robot wants (task, waypoint, preferred direction, priority) without ever touching collision safety:



The invariant this document keeps coming back to: **everything above the line only reorders or restricts candidate moves; only the PIBT recursion (§7) and `validate.py` (§7.4) decide collision-freedom.** No high-level formula can, by construction, produce two robots on the same vertex.

1. Graph and warehouse model (`graph.py` , `warehouse.py`)

In one sentence. The warehouse is a grid of open and blocked cells. This section defines how far apart two cells are, which cells are corridors, which are junctions, and which single cells the whole map depends on.

1.1 Grid graph

`GridGraph` is a 4-connected grid: from $v = (r, c)$, the neighbor set is

$$N(v) = \{ (r \pm 1, c), (r, c \pm 1) \} \cap \text{passable-cells}$$

(`_DELTA`s = $((-1, 0), (1, 0), (0, -1), (0, 1))$). Degree $\deg(v) = |N(v)|$; the max over the graph is $\Delta(G) = \max_v \deg(v)$ (`GridGraph.max_degree`), which appears in the PIBT complexity bound (§7.5).

1.2 BFS distance maps

For a goal `g`, `compute_bfs_distance_map(g)` runs a reverse-BFS from `g` and returns

$$D_g(v) = d_{\text{route}}(v, g) \quad (\text{graph-shortest-path hop count; INF if unreachable})$$

for every vertex `v`, in $O(|V|)$ since the grid is unweighted. This map is cached per goal (`distance_map`) and is the single most-reused primitive in the codebase — it backs route planning, progress scoring, congestion lookahead, task-assignment cost, and the A* heuristic for both the direction-aware router (§8) and the baseline planners (§14), all without recomputation. `Warehouse.precompute()` eagerly builds `D_g` for every pickup/delivery/parking vertex.

`shortest_route(source, goal, horizon=None)` walks `D_g` greedily: at each step, move to the neighbor `n` minimizing `D_g(n)` (ties broken by a fixed neighbor iteration order, so the result is deterministic). An optional `horizon` truncates the returned route after that many steps — used when only a lookahead prefix is needed (e.g. congestion's `downstream`, §5.3).

1.3 Structural features

Computed once via an **iterative Hopcroft–Tarjan lowlink DFS** (`_compute_articulation_and_bridges`, non-recursive to avoid Python's recursion limit on large maps):

- **Articulation points:** vertices whose removal disconnects the graph.
- **Bridges:** edges whose removal disconnects the graph — i.e. edges that lie on no cycle.
- **Dead ends:** $\{ v : \deg(v) \leq 1 \}$.
- **Intersections:** $\{ v : \deg(v) \geq 3 \}$ — these are exactly the vertices that separate one aisle from the next (§1.4).

`satisfies_pibt_reachability() = (dead_ends =  $\emptyset$ ) \wedge (bridges =  $\emptyset$ )` — a rough check of PIBT's theoretical sufficient condition that every edge lie on a simple cycle (needed for PIBT's progress guarantee; see §7.5's caveat).

`bfs_ball(center, radius)` is a plain BFS expansion returning every vertex within `radius` graph hops of `center` — used for deadlock spatial clustering (§10.3) and escape-vertex search (§10.4).

1.4 Aisle segmentation

An **aisle** is a maximal connected component of the graph after removing all intersection vertices ($\deg \geq 3$) — i.e. the corridors *between* intersections. Each aisle's `vertices` list is ordered from one endpoint to the other (`_order_component`: find a local-degree- ≤ 1 endpoint, then walk neighbor-to-neighbor). Moving to a higher index is **FORWARD**; to a lower index, **REVERSE** (`AisleDirection`):

```
entry_direction(v) = FORWARD if index(v) ≤ (length-1)/2
                   = REVERSE  otherwise
```

i.e. entering nearer the start implies you're heading forward through it.

Aisle capacity, `_aisle_capacity(length)`, has two models (`Params.aisle_capacity_model`):

```
"length" (default):    capacity = clamp( ceil(ratio · length),          1, aisle_capacity )
"throughput":          capacity = clamp( ceil(ratio · length / T_min), 1, aisle_capacity )
```

where `ratio = aisle_capacity_ratio` (default 1.0) and `T_min = minimum_aisle_lock_time` (default 20). The rationale for the `"throughput"` alternative: a single-file corridor is throttled by how fast the robot at its *exit* can leave, not by how many robots physically fit inside — packing it to `length` just builds a queue that can never drain (this is diagnosed empirically in the README's "Implementation notes" cause 1).

Each aisle's `minimum_lock_time = max(2, min(minimum_aisle_lock_time, 2·length))` — capped so a short aisle doesn't lock disproportionately long relative to its own size.

Manageability (`_mark_manageable_aisles`): an aisle is permanently **OPEN** (never given a one-way direction) if either

```
length < directional_aisle_min_length    (default 4; too short to be worth a rule)
OR
some edge of the aisle (or its two boundary edges to adjacent intersections)
is a bridge                             (one-way-ing a cut edge disconnects the graph)
```

2. Robots and tasks (`robot.py` , `task.py`)

In one sentence. What a robot remembers from one timestep to the next, and where the never-ending stream of jobs comes from.

2.1 Robot state

Robot carries, among other fields: `priority` (float, recomputed every step, §4), `tie_breaker = 1e-4 · id` (deterministic, unique, and small enough to never reorder two robots across a priority-class boundary), `waiting_time` / `blocked_time` / `no_progress_steps` (three related but distinct stall counters — see §10.1 for how they differ), `route` (the current planned path, a `List[Vertex]`), `route_distance_to_waypoint (=  $D_w(x_i(t))$  , the live BFS distance to the current waypoint), mode (a ProximityMode , §4.1), direction_weight / aisle_weight (  $\beta_i / \gamma_i$  , §4.2–4.3), and position_history (a deque(maxlen=16) used for repeated-configuration detection, §10.2).`

2.2 Task urgency and waiting

```
urgency(t)      = 0                                if deadline = INF
                 = 1 / max(1, deadline - t) otherwise
waiting_time(t) = max(0, t - release_time)
service_time    = completion_time - release_time    (only once completed)
```

`urgency` grows without bound as the deadline approaches and is passed (`slack` $\rightarrow 0$ or negative), which is intentional: a task past its deadline should dominate the priority function's urgency term (§4) rather than saturate.

2.3 Arrival processes (`TaskGenerator`)

Four modes, each returning a task count for `receive_new_tasks(t)` :

```
periodic: count = rate                if t mod period = 0, else 0
poisson:  count = Poisson(rate)        (via Knuth's sampler, below)
bursty:   count = burst_size           if t mod burst_period = 0, else 0
batch:    count = total                 if t = 0, else 0
```

Knuth's Poisson sampler (`_poisson`), the standard inverse-transform-free method for small-to-moderate λ :

```
L = e^{-λ}; k = 0; p = 1
repeat: p ← p · U          where U ~ Uniform(0,1)
        if p ≤ L: return k
        k ← k + 1
```

Correctness sketch: after k multiplications, $p = \prod U_1 \dots U_k$, which is distributed as $\text{Gamma}(k, 1)$'s survival function evaluated at a uniform draw; the number of uniform draws needed to push the running product below $e^{-\lambda}$ is exactly Poisson-distributed with mean λ . This is the textbook algorithm (Knuth, *The Art of Computer Programming*, Vol. 2). Guarded at $k > 1000$ for numerical safety, though this never triggers at realistic rates.

3. Priority function (`priority.py`)

In one sentence. Who plans first each timestep. Robots are ranked by what they are carrying, and a robot that has been waiting long enough always eventually outranks everyone — that bound is the fairness guarantee.

The priority function must satisfy two competing goals: a loaded robot should almost always outrank a free one (so deliveries don't stall behind idle repositioning), yet **no robot should starve forever**. LDA-PIBT achieves both with a class term plus an unbounded linear-in-waiting term:

$$p_i(t) = P_{\text{class}}(i) + k_w \cdot W_i + k_b \cdot B_i + k_u \cdot U_i + k_e \cdot E_i + \epsilon_i$$

TERM	MEANING	CODE
<code>P_class(i)</code>	task-class base priority (below)	<code>task_class_priority</code>
<code>k_w · W_i</code>	<code>waiting_weight · robot.waiting_time</code>	
<code>k_b · B_i</code>	<code>blocked_weight · robot.blocked_time</code>	
<code>k_u · U_i</code>	<code>urgency_weight · task.urgency(t)</code> (0 if no task)	<code>compute_task_urgency</code>
<code>k_e · E_i</code>	<code>priority_inside_aisle</code> if <code>current_aisle</code> is not <code>None</code> else 0	
<code>ε_i</code>	<code>robot.tie_breaker = 1e-4 · id</code>	

3.1 Task-class priority

```
P_class(i) = priority_emergency    (400)    if robot.state = RECOVERY
             = priority_repositioning (100)   if no task, state = PARKED
             = priority_free         (0)      if no task, otherwise
             = priority_loaded       (300)   if task.status = TO_DELIVERY
             = priority_pickup       (200)   if task.status = TO_PICKUP
             = priority_free         (0)      otherwise
```

giving the ordering `P_emergency` (400) > `P_loaded` (300) > `P_pickup` (200) > `P_repositioning` (100) > `P_free` (0) — a robot mid-deadlock-recovery always outranks everything else; a robot carrying a delivery outranks one still travelling to pick up; an idle robot moving to a parking bay slightly outranks a plain idle one (so it isn't shoved off its chosen bay by another idle robot).

3.2 Fairness horizon

Because `k_w · W_i` grows without bound while `P_class` is fixed, any robot eventually outranks any other, however low its class. `fairness_horizon` computes exactly how long that takes:

```
fairness_horizon = (priority_emergency - priority_free) / waiting_weight
                  = (400 - 0) / 5.0 = 80    [default params]
```

i.e. after waiting 80 steps, a `priority_free` robot's $k_w \cdot W_i$ term alone exceeds the entire class spread and it will outrank even an emergency robot with $W = 0$. This is exactly the guarantee `test_lifelong.py` checks (`test_waiting_eventually_overcomes_the_class_gap`, using `waiting_time = 500 >> 80`). If `waiting_weight ≤ 0`, this returns $+\infty$ — fairness would no longer be guaranteed, which the function surfaces rather than silently allows.

Worked example. The same scenario run out to timestep 149, so that robots have had time to accumulate waiting. Three of the 24: the highest-priority robot, the one that has waited longest, and the lowest-priority one.

robot	state	task	P_class	$k_w \cdot W$	$k_b \cdot B$	$k_e \cdot E$	$p_i(t)$
robot 15	to_delivery	to_delivery	300	0	0	50	350.002
robot 3	to_pickup	to_pickup	200	15	0	50	265
robot 0	to_pickup	to_pickup	200	0	0	50	250

`P_class` sets the band; the waiting and blocked terms move a robot within it and, given long enough, out of it. The full class spread is 400 (400 for a robot in recovery down to 0 for an idle one) and $k_w = 5$, so 80 steps of waiting is worth the entire spread: a robot that waits that long outranks *any* robot of any class (§3.2).

That bound is the fairness guarantee, and it is the reason the priority function has a waiting term at all. Without it a permanently higher-class neighbour — a loaded robot on a floor that always has loaded robots — could starve an idle one indefinitely, and the throughput number would never show it.

3.3 Tie-breaking and ordering

`order_by_priority` sorts by `(-priority, id)` — descending priority, id as a deterministic secondary key. Combined with the $\epsilon_i = 1e-4 \cdot id$ term baked into `priority` itself, ties are essentially never hit in practice, but the explicit `id` key guarantees determinism regardless.

4. Candidate scoring (`scoring.py`)

In one sentence. Given the moves a robot is *allowed* to make, this is the single number that ranks them. Every preference in the system — direction, congestion, turning cost, aisle rules — is finally cashed out here, and nowhere else.

Once PIBT (§7) has produced a *legal* candidate set for a robot, something must rank them — that ranking is what turns "collision-free" into "collision-free *and* purposeful". This is entirely separate from legality: scoring only reorders candidates that already passed every hard rejection rule in §7.2.

4.1 Proximity mode

A robot's behavior should change as it nears its target: far away, only progress and direction matter; up close, don't zigzag chasing a one-step-shorter path. Route distance $r_i = D_w(x_i)$ is bucketed against two thresholds $R_{near} = 2$, $R_{far} = 8$:

```
mode(r_i) = TRANSIT   if r_i > R_far
           = APPROACH  if R_near < r_i ≤ R_far
           = ARRIVAL   if r_i ≤ R_near
```

4.2 Smooth direction weight β_i

Rather than a step function on `mode`, the weight on "did this move match my preferred direction" decays smoothly as the robot approaches:

```
 $\beta_i = \beta_{\max} \cdot \min(1, \max(0, r_i - R_{\text{near}}) / \max(1, R_{\text{far}} - R_{\text{near}}))$ 
```

with $\beta_{\max} = \text{beta_strong} = 3.0$. At $r_i \geq R_{\text{far}}$, $\beta_i = \beta_{\max}$ (full strength, TRANSIT-like); at $r_i \leq R_{\text{near}}$, $\beta_i = 0$ (no directional preference at all — free to approach from any angle). $\beta_i = \text{beta_strong}$ unconditionally when $r_i = \text{INF}$ (route unreachable: keep whatever direction preference was already computed rather than collapsing it), and 0.0 when `direction_control == "none"`.

4.3 Aisle-continuity weight γ_i

A discrete analogue of the same idea, for the reward on staying in the current aisle rather than cutting toward an intersection early:

```
 $\gamma_i = \text{gamma\_strong}$  (2.0) if mode = TRANSIT  
= (gamma_strong + gamma_weak) / 2 (1.25) if mode = APPROACH  
= gamma_weak (0.5) if mode = ARRIVAL  
= 0 if direction_control = "none"
```

4.4 Turning cost

Penalizes changing heading between consecutive moves, with a heavier penalty for a full reversal:

```
P_turn(previous, movement) = 0 if turning_cost disabled, or either move is STAY  
= 0 if movement = previous (straight)  
=  $\lambda_{\text{reverse}}$  if movement = previous.opposite() (180° reversal, default 2.0)  
= 1 otherwise (90° turn)
```

4.5 Progress

The core "am I getting closer" signal, with two normalizations (`Params.progress_normalization`):

```
d_cur = D_w(x_i) (current distance to waypoint)  
d_cand = D_w(candidate) (candidate's distance to waypoint)  
 $\Delta = d_{\text{cur}} - d_{\text{cand}}$   
  
"route" (spec-literal): progress =  $\Delta / \max(1, d_{\text{cur}})$   
"step" (default): progress =  $\Delta \in \{-1, 0, +1\}$  on a grid
```

Edge cases: if the candidate is unreachable ($d_{\text{cand}} = \text{INF}$), $\text{progress} = -1$; if the *current* position is unreachable but the candidate is reachable, $\text{progress} = 0$ (treated as neutral rather than an artificially huge gain). "step" is the default rather than the literal spec formula because at large d_{cur} the "route" normalization shrinks $\alpha \cdot \text{progress}$ below β_{strong} and λ_{turn} (e.g. at $d_{\text{cur}} = 20$, $\alpha \cdot \text{progress} \approx 10/20 = 0.5 < 3.0 = \beta_{\text{strong}}$), silently inverting the intended dominance ordering (§4.6) — see `docs/implementation-notes.md` deviation 1.

4.6 The full candidate score $S_i(v)$

```
S_i(v) =  $\alpha$  · progress
        +  $\beta_i$  · 1[move  $\neq$  STAY  $\wedge$  move = preferred_direction]
        +  $\gamma_i$  · 1[candidate stays in the current aisle, and robot is not at an intersection]
        -  $\lambda_{\text{turn}}$  · P_turn
        -  $\mu$  · C_i(v)                (congestion, §5)
        -  $\nu$  · P_wait
        -  $\xi$  · 1[candidate is a bottleneck vertex]
        -  $\zeta_{\text{counterflow}}$  · 1[move opposes the aisle's committed direction]    (§6.6)
        -  $\zeta_{\text{reservation}}$  · 1[entering a directional aisle without a ticket]    (§6.5)
```

Defaults: α = alpha_progress = 10.0, λ_{turn} = 0.5, μ = mu_congestion = 1.0, ν = nu_wait = 0.2, ξ = xi_bottleneck = 1.0, $\zeta_{\text{counterflow}}$ = $\zeta_{\text{reservation}}$ = 8.0. P_wait = 1 unless the candidate is "stay in place while already at the waypoint" (free) — otherwise waiting is always mildly discouraged so a robot with any legal forward move prefers it.

The intended weight ordering is $\alpha > \zeta > \beta_{\text{strong}} > \gamma_{\text{strong}} > \lambda_{\text{turn}}$ (10 > 8 > 3 > 2 > 0.5): a whole step of progress beats everything, driving the wrong way down an aisle beats every other preference, and a single turn is the cheapest term. Two things have to hold for that ordering to be real:

- $\zeta < \alpha$. The two ζ terms are where the aisle layer acts, and they are *penalties*, not rejections. Until this pass they were hard rejections in `feasible_candidates`, which deletes legal moves from the candidate set — and PIBT's progress argument rests on being able to push any robot into an adjacent cell (§7.4). Pricing counterflow instead keeps that freedom: a robot drives the wrong way when that is the only way to make progress, or when nothing else is left, and pays for it. `hard_direction_constraints` restores the old behaviour for comparison.
- $\mu \cdot C_i(v) \leq \mu \cdot C_i(v)$ has to be normalised (§5.3) or the mixture is a robot *count*, and $\mu \cdot C$ then reaches the scale of $\alpha \cdot \Delta$ — measured at mean 3.40, p90 5.75, max 9.60 on `warehouse_medium` with 40 robots — which makes congestion the second-largest term in the score rather than a modulator of the smaller ones.

The "route" progress normalization (§4.5) breaks the ordering at long range for a third reason, which is why "step" is the default.

`CandidateScorer.sort_key(robot, v) = (-score(robot, v), v)` — descending score, ties broken by vertex coordinates for determinism.

Worked example. `warehouse_medium`, variant `full_lda_pibt`, seed 7, 24 robots, timestep 18. Robot 3 is at (10,20) heading for waypoint (0,12), 18 steps away, so it is in `TRANSIT` mode with $\beta_i = 3$ and $\gamma_i = 2$ (§4.2, §4.3). Its previous move was `south` and its preferred direction is `north`. Every column below is computed by the same functions `CandidateScorer.score` itself calls, and they sum to the printed $S(v)$:

cell	move	$\alpha \cdot \text{prog}$	$\beta \cdot \text{dir}$	$\gamma \cdot \text{aisle}$	$-\lambda \cdot \text{turn}$	$-\mu \cdot \text{cong}$	$-\nu / -\xi$	$-\zeta$	$S(v)$	rejected by	priced
(9,20)	north	+10	+3	+0	-1	-0.11	+0	+0	11.89	vertex-conflict	-
(10,19)	west	+10	+0	+0	-0.5	-0.1	+0	+0	9.4	-	-
(10,20)	stay	+0	+0	+0	+0	-0.16	-0.2	+0	-0.36	-	-
(10,21)	east	-10	+0	+0	-0.5	-0.16	+0	+0	-10.66	-	-
(11,20)	south	-10	+0	+0	+0	-0.28	+0	-16	-26.28	-	aisle-direction, no-reserv

Read the last two columns first, because they are different kinds of thing. **Rejected by** is legality: those moves are gone before scoring, and no score could rescue them. **Priced** is preference: the move is still on the table, it just costs.

(9,20) scores highest at 11.89 and is unavailable regardless: `vertex-conflict` removed it in §7.2. So the move actually chosen is (10,19) at $S = 9.4$ — the best of what is *legal*.

The shape of the winning number is the whole argument: progress contributes +10, while every other term together moves it by -0.6. The preference terms break ties between moves that agree on progress; they never outvote progress itself. That is what $\alpha > \zeta > \beta > \gamma > \lambda$ buys.

(11,20) is the instructive row. It pays 16 for `aisle-direction, no-reservation` and finishes 35.68 behind — last by a wide margin, and still in the candidate set. Set `hard_direction_constraints` and that same move is deleted instead; a move PIBT cannot see is a move priority inheritance cannot use to unblock a chain (§7.4). The gap between those two treatments is what the README's ablation measures as 1.9× to 3.1× throughput.

4.7 Preferred route direction and hysteresis

```
compute_route_direction(robot):
    if len(route) ≥ 2: return movement_direction(route[0], route[1])
    else: pick the neighbor n maximizing D_w(position) - D_w(n)    (steepest single-step descent)
```

`apply_direction_hysteresis` keeps the *previous* preferred direction unless one of: hysteresis is off, the previous preference was `STAY`, the proposed direction already matches, the robot has been blocked $\geq t_{\text{blocked}}$ steps, the robot isn't in `TRANSIT` mode, or the robot is currently at an intersection (`current_aisle` is `None`) — i.e. **new direction commitments only happen at decision points**, preventing a robot from flip-flopping mid-corridor as `compute_route_direction`'s greedy choice wobbles.

5. Congestion model (`congestion.py`)

In one sentence. How crowded a cell is about to be, expressed as one number between 0 and 1 so that it can modulate the score without ever dominating it.

5.1 Occupancy index

`OccupancyIndex` is rebuilt every timestep in $O(n)$ (n = robot count): a `vertex` $\rightarrow$ `robot` hash map, an `aisle_id` $\rightarrow$ `count` counter, and a **spatial hash** (`bucket_counts`) with cell size = $\max(1, \text{local_congestion_radius})$ for fast local-density queries.

```
aisle_load(a) = occupancy(a) / max(1, capacity(a))    (a normalized  $\geq 0$  congestion ratio, can exceed 1)
```

`local_density(v, exclude)` approximates $|\{j : d_{\text{Manhattan}}(v, x_j) \leq R_{\text{local}}\}|$ (excluding one robot, typically the querying robot itself) in two passes: first sum the 3×3 bucket window around `v`'s bucket for a cheap upper bound; if nonzero, refine exactly by scanning the Manhattan diamond of radius `R_local` = `local_congestion_radius` (default 3) — for each `dr` $\in [-R, R]$, scan `dc` over `span = R - |dr|`.

5.2 Downstream lookahead

```
downstream(v, w) = mean_{u  $\in$  tail} 1[occupied(u)]
```

where `tail = shortest_route(v, w, horizon=K)[1:]` and `K = downstream_horizon` (default 5) — the fraction of the next `K` route vertices beyond `v` that are currently occupied. Cached per `(v, w)` per timestep (`begin_timestep()` clears the cache) since many robots query the same downstream segment.

5.3 Congestion mixture

```
C_i(v) = (  $\omega_{\text{local}} \cdot \text{local\_occupancy\_ratio}(v)$ 
          +  $\omega_{\text{aisle}} \cdot \text{aisle\_load}(\text{aisle}(v))$ 
          +  $\omega_{\text{downstream}} \cdot \text{downstream}(v, w_i)$  ) / ( $\omega_{\text{local}} + \omega_{\text{aisle}} + \omega_{\text{downstream}}$ )
```

All three weights default to `1.0`, so `C_i(v)` is the mean of three ratios and lies in $[0, 1]$. Returns `0` unconditionally if `congestion_scoring` is off.

`local_occupancy_ratio(v) = local_density(v) / |\{u  $\in$  V :  $d_1(u, v) \leq R_{\text{local}}$ \}|` — the fraction of reachable cells within the radius that hold a robot, not the robot count. `C_aisle` and `C_downstream` were already ratios, so mixing a count with them made `C_local` effectively the whole mixture and let $\mu \cdot C$ outrank the terms it is meant to modulate (§4.6). `congestion_normalisation=False` restores the raw-count form.

Normalising is a *calibration* fix, not a performance one: measured on the bundled maps it is neutral to slightly negative on throughput. What it buys is that the weight ordering the score claims is the ordering it has.

Worked example. The same robot and timestep. `C_i(v)` mixes three occupancy signals with weights `$\omega_{\text{local}} = 1$` , `$\omega_{\text{aisle}} = 1$` , `$\omega_{\text{down}} = 1$` , then divides by their sum (3) because `congestion_normalisation` is on:

cell	<code>C_local</code>	<code>C_aisle</code>	<code>C_down</code>	<code>C_i(v)</code>	$-\mu \cdot C$
(9,20)	0.091	0.25	0	0.114	-0.114
(10,19)	0.091	0	0.2	0.097	-0.097
(10,20)	0.077	0	0.4	0.159	-0.159
(10,21)	0.091	0	0.4	0.164	-0.164
(11,20)	0.182	0.25	0.4	0.277	-0.277

Normalisation is what makes this a modulator rather than a rival. The largest `C_i(v)` here is 0.277, so the entire congestion term is worth at most 0.277 — against $\alpha = 10$ for a single step of progress. Turn normalisation off and `C_local` reverts to a raw robot count: $\mu \cdot C$ then reaches the scale of $\alpha \cdot \Delta$ (measured mean 3.40, p90 5.75, max 9.60 on this map at 40 robots), and congestion stops modulating the smaller terms and starts overruling the largest one.

5.4 Route-level congestion (for task assignment)

```
route_congestion(source, goal) = occupied_count / len(route)
                               + (  $\sum_{a \in \text{distinct aisles on route}} \text{aisle\_load}(a)$  ) / max(1, #distinct aisles)
```

Used only by `assignment.py`'s cost function (§9), not by per-step candidate scoring.

6. Aisle management (`aisle_manager.py`)

In one sentence. Narrow corridors get a traffic light. Robots ask for a direction, the corridor decides from the aggregate, and it holds that decision long enough for the decision to be worth making — but not forever.

Design principle, stated verbatim in the code twice: "**robots generate directional requests, but aisles make directional decisions.**" No single robot can force an aisle to flip; direction is a property the aisle decides from the *aggregate* of what's asking.

6.1 Requested direction

`requested_direction(robot, aisle)` projects the robot's planned route onto the aisle's own index ordering: if the route touches the aisle at ≥ 2 points, `FORWARD` if the index increases, `REVERSE` if it decreases; for a single touching vertex, falls back to `aisle.entry_direction(v)` (§1.4).

6.2 Per-robot directional demand

$$\text{demand}_i(a) = w_u \cdot U_i + w_w \cdot W_i + w_p \cdot P_i - w_l \cdot L_i - w_c \cdot C_i$$

SYMBOL	CODE	MEANING
<code>U_i</code>	<code>task.urgency(t)</code> if the robot has a task, else 0	deadline pressure
<code>W_i</code>	<code>max(waiting_time, no_progress_steps) + blocked_time</code>	stalled route progress also counts as waiting — a robot shuffling in place without a route-distance gain still "starves" and should be able to flip the aisle
<code>P_i</code>	<code>1 / (1 + min(d(x_i, start), d(x_i, end)))</code> (0 if unreachable)	proximity to the nearer aisle endpoint — a robot already at the door counts more than one still approaching
<code>L_i</code>	<code>route_distance_to_waypoint</code> (0 if INF)	route length remaining — long-route robots are discounted since flipping doesn't help them as urgently
<code>C_i</code>	<code>aisle_load(a)</code>	already-present congestion discourages adding more demand

Default weights: `w_u = 1.0`, `w_w = 0.5`, `w_p = 2.0`, `w_l = 0.05`, `w_c = 0.5`.

6.3 Aggregate demand and the decision rule

```
S_a^+ = Σ_{i requesting FORWARD in aisle a} p_i(a) · demand_i(a)
S_a^- = Σ_{i requesting REVERSE in aisle a} p_i(a) · demand_i(a)
imbalance = S_a^+ - S_a^- [+ parity_bias · (+1 if a even else -1), if parity_bias ≠ 0 and either sum is nonzero]
```

`p_i(a)` is the attribution weight. By default each robot is charged to exactly one aisle (`p = 1` for `next_aisle`, else `current_aisle`; 0 elsewhere). With `demand_spread` on, `update_aisle_queues` charges the robot to *every* aisle its route touches, with `p_i(a) = 1 / (1 + steps to the entry)` — the spec-12 reading, and the only one that lets an aisle see both the robots already inside it and traffic more than one aisle away. Measured on the bundled maps it engages

direction control considerably more often and costs throughput, so it ships off by default; the flag exists so the measurement stays reproducible.

`update_aisle_direction` then applies **hysteresis with a dead band** `[- $\tau_{\text{switch}}$ ,  $\tau_{\text{switch}}$ ]`, `$\tau_{\text{switch}}$  = direction_switch_threshold` (default 5.0, applied only if `hysteresis` is on, else 0), **and a maximum green:**

```
occupied, holding FORWARD: DRAINING if imbalance < - $\tau_{\text{switch}}$ 
                           or (t - direction_since  $\geq$  T_max and S_a^- > 0)
occupied, holding REVERSE: DRAINING if imbalance > + $\tau_{\text{switch}}$ 
                           or (t - direction_since  $\geq$  T_max and S_a^+ > 0)
empty, within lock_until: hold current direction (minimum-lock enforcement)
empty, past lock:
    imbalance >  $\tau_{\text{switch}}$  (and forward demand > 0, or  $\tau$  = 0): commit FORWARD
    imbalance < - $\tau_{\text{switch}}$  (and reverse demand > 0, or  $\tau$  = 0): commit REVERSE
    else: OPEN / NONE
```

`T_max = maximum_aisle_lock_time` (default 40), floored at the aisle's own minimum lock so the two clocks cannot contradict each other.

Why a maximum green is necessary. Hysteresis is only half a traffic signal. It bounds how *soon* a committed direction may change and says nothing about how long it may persist. Consider a warehouse with pickups down one side and deliveries down the other — the standard layout, and three of the four bundled maps: loaded robots want one direction and empty ones want the other, in near-equal measure, so `|imbalance|` sits inside the dead band indefinitely. Under the dead band alone an aisle therefore holds its first committed direction for the rest of the run and half its traffic never gets through. Instrumented on `warehouse_corridors`: all 35 robots stalled by `t = 120`, four aisles locked REVERSE with `lock_until` long expired, and the identical state still present at `t = 199`.

The maximum green makes the aisle *starvation-free*: a direction held for `T_max` steps with any opposing demand at all must drain and flip. Flips forced this way are counted separately as `starvation_flips`, which keeps the H2 question ("does hysteresis stop oscillation?") distinct from the liveness one.

`_commit` bumps `direction_switches` (both aisle-level and manager-level) only on an *actual* change from a non-NONE previous direction, and sets `lock_until = t + minimum_lock_time` and `direction_since = t`, so a freshly committed direction can neither be reversed for `minimum_lock_time` steps nor held past `maximum_lock_time` against opposing demand — checked by `test_minimum_lock_time_blocks_an_immediate_flip` and `test_maximum_green_forces_a_flip_under_balanced_demand`.

Worked example. Same run, timestep 18. Aisle 38 runs (11,20) -> (14,20) (length 4, capacity 4, axis `col`). Each robot routed through it contributes $w_u \cdot U + w_w \cdot W + w_p \cdot P - w_l \cdot L - w_c \cdot C$ (§6.2) to whichever direction it wants, and the two sides are summed:

```

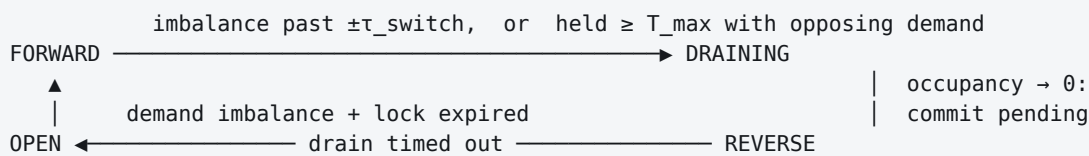
S_a^+ (forward demand)    = -0.558
S_a^- (reverse demand)    = 4.758
imbalance S_a^+ - S_a^-    = -5.317
dead band ± τ_switch       = 5
occupancy                  = 1 robot(s) inside
state on entry             = REVERSE, direction REVERSE
locked until t             = 26 (T_min = 8, T_max = 40)

```

With hysteresis on, `update_aisle_direction` returns `REVERSE`; with the dead band removed it returns `REVERSE`. The two agree here, which is the common case: hysteresis is not meant to fire often, it is meant to make the rare flip deliberate.

An imbalance of 5.317 against a dead band of 5 clears the band, and a committed direction is additionally locked for `T_min` = 8 steps. The maximum green matters for the opposite failure: past `T_max` = 40 steps *any* opposing demand forces a drain, so a balanced aisle - pickups on one side, deliveries on the other, imbalance permanently inside the band - cannot hold one direction forever and starve the traffic wanting the other way.

6.4 Aisle state machine



Entering `DRAINING` records `pending_direction` = the opposite of the current one. When the aisle empties, that direction is committed **directly**, without re-running the imbalance test: the demand that triggered the drain has usually moved on by the time the aisle is clear, so re-testing would re-commit the direction just drained and the flip would never happen.

Plus one addition beyond the base rule: a `DRAINING` aisle that has not reached zero occupancy within `max_drain_time` (default 30) steps is force-reopened to `OPEN` — justified as "the current direction is infeasible" (an absorbing deadlock otherwise: an aisle that can never empty would never leave `DRAINING`).

6.4.1 Coordinated direction assignment

`coordinate_aisle_directions` (default off) decides the directions as a *set* rather than one aisle at a time: aisles commit in descending `|imbalance|`, and any commit that would break strong connectivity of the directed residual graph is rolled back, leaving that aisle bidirectional. The check is a two-way BFS over the passable cells under the current assignment, $O(|V| + |E|)$ per commit.

This was the fix the earlier review expected to matter most. It does not: on the bundled maps its measured effect on throughput is between ± 0.000 and ± 0.011 , because a single one-way aisle almost never disconnects a ladder graph, so the guard hardly ever fires. The collapse it was meant to cure was a liveness failure (§6.3), not a connectivity failure. It ships as a flag so the null result is reproducible.

6.5 Reservations

`update_aisle_reservations` first expires any `Reservation` with `t > expiry_time`, then grants a fresh one to a robot **only if all of**: the aisle isn't `DRAINING`; the robot's requested direction matches the aisle's current direction (or the aisle has none yet); the robot doesn't already hold one; `occupancy + (pending reservations held by robots`

still outside) < capacity ; and the robot is `_at_entrance` — within `route_distance ≤ 3` of either endpoint. A granted reservation has

```
expected_exit_time = t + floor(length) + 1
expiry_time        = t + reservation_ttl    (default 15)
```

`can_enter_aisle` (the read side, consumed by the ζ _reservation term of §4.6) combines all of this into two separate gates, because they answer different questions:

- the **occupancy cap** applies to every managed aisle, `OPEN` included. Over-filling a single-file corridor is what builds a queue that cannot drain, and that is true whether or not the corridor currently has a direction. This is the mechanism H3 is about.
- the **ticket** is required only once the aisle is directional, per spec 18. Demanding one to enter an `OPEN` aisle throttles every crossing on a map whose aisles are mostly short and mostly open.

```
DRAINING                → False
state ≠ OPEN and direction mismatch → False
occupancy ≥ capacity    → False
reservations off, or state is OPEN → True
otherwise                → has_valid_reservation
```

Gating. `update_aisle_reservations` and `violates_aisle_reservation` are gated on `params.reservations` alone. They used to be gated on `AisleManager.enabled ( direction_control == "aisle" )`, which made the flag a silent no-op in every robot-level configuration — including `reservations_only`, the variant that exists specifically to isolate it, whose runs were bit-identical to its own control. `AisleManager.active ( enabled or reservations )` is what `Simulator.step` now uses to decide whether to run the layer at all.

6.6 Movement legality (spec 27, `violates_aisle_direction`)

The one deliberate deviation here: direction restricts **entry**, not **interior movement**. A robot already inside a directional aisle may always make an **egress move** toward the nearer endpoint:

```
is_egress_move(current, candidate):
    let ci = index(current), ni = index(candidate), last = length - 1
    return min(ni, last - ni) < min(ci, last - ci)
```

i.e. the candidate is strictly closer (in index-distance) to *whichever* endpoint is nearest, regardless of which one that is. Without this, a `DRAINING` aisle could trap a robot facing the "wrong" way and the aisle would never actually empty.

7. PIBT core (`pibt.py`)

In one sentence. The only part of the system that decides collision-freedom. A robot takes the best move it can; if the cell it wants is occupied, it lends its priority to the occupant and asks it to move first.

This is the safety-critical layer: everything above only ranks and restricts candidates, but only this recursion (plus `validate.py`, §7.4) decides what is *collision-free*.

7.1 Candidate set

```
C_i = N(x_i) ∪ { x_i }
```

— every grid neighbor, plus staying in place. Staying is always in the set, which is what guarantees the recursion always terminates with *some* move (possibly `INVALID`, meaning "stay put", never "no answer").

7.2 Hard rejection rules (applied in this exact order)

1. **Off-graph:** `candidate ∉ V`.
2. **Kinematics:** `violates_kinematics` — currently a stub, always `False`. A hook for future footprint/turning-radius constraints (`Robot.orientation` exists but is unused for this).
3. **Vertex conflict** (§7.3.1).
4. **Swap conflict with the calling parent** (§7.3.2), only checked when this call is a recursive priority-inheritance request (`parent ≠ None`).

Surviving candidates are then **sorted by score** (§4.6, descending) — legality and preference are fully decoupled: scoring never overrides a rejection, and rejection never depends on score.

Aisle direction and reservations are deliberately not on this list. Every rule here concerns safety or physical possibility; the aisle layer concerns *preference*, and it acts through the two ζ terms of §4.6 instead. The distinction is load-bearing rather than stylistic: §7.4's progress argument assumes a robot can always be pushed into an adjacent cell, and a one-way rule enforced by rejection removes exactly that. With aisle direction as a hard rejection, inheritance chains dead-end against the constraint and the aisle layer collapses — measured at 0.010 tasks/step against 0.153 for the same configuration with the same direction decisions priced instead of enforced.

`hard_direction_constraints=True` reinstates rules 3 and 4 of the old order (aisle direction, then reservation, before the conflict checks) and suppresses the ζ terms so the penalty is not counted twice.

7.3 Conflict checks (`validate.py` companions, spec 26)

Vertex conflict, $O(1)$ via the reservation set:

```
creates_vertex_conflict(robot, candidate) =  
    candidate ∈ reserved_vertices                                (already tentatively claimed this step)  
    ∨ (∃ occupant at candidate, occupant ≠ robot, occupant.next_position = candidate)  (occupant already committed there)
```

Swap conflict (a 2-cycle position exchange):

```
creates_swap_conflict(robot, parent, candidate) =  
    candidate = parent.position ∧ parent.next_position = robot.position
```


7.4 The PIBT recursion (spec 25)

```
pibt(robot, parent, t):
    feasible ← feasible_candidates(robot, parent, t)    # legality-filtered, score-sorted
    for candidate in feasible:
        if candidate already reserved this step: skip
        tentatively set robot.next_position = candidate; reserve it
        occupant ← whoever currently sits at candidate
        if occupant is None or occupant = robot:
            return VALID                                # empty cell, or staying put
        if occupant.next_position is not None:
            if occupant.next_position ≠ candidate:
                return VALID                            # occupant is moving away from candidate – safe to follow
            else:
                undo reservation; robot.next_position ← None; continue    # occupant is also staying there – conflict
        # occupant has no committed move yet: ask it to move (priority inheritance)
        robot.waiting_for_robot ← occupant
        result ← pibt(occupant, robot, t)                # recurse
        if result = VALID:
            return VALID
        # occupant could find no legal move at all: backtrack
        undo reservation; robot.next_position ← None; continue
    # every candidate failed: stay in place (always legal, since x_i ∈ C_i)
    robot.next_position ← robot.position; reserve it
    return INVALID
```

`plan_step` calls `pibt(robot, None, t)` for every robot in **descending priority order** (§3.3), skipping any robot whose `next_position` was already set by an earlier robot's recursive chain. This is the mechanism that makes priority meaningful: a low-priority robot occupying a high-priority robot's preferred cell gets *asked* to move (via the recursive call), and if it can find any legal alternative, both robots move without conflict — a chain of such requests is what "inheritance" refers to. `INVALID` (a robot stuck in place) increments `blocked_time`, which feeds back into both the priority function (§3) and deadlock detection (§10).

Statistics tracked: `recursive_calls` (recursion depth proxy), `backtracks` (times an inheritance chain failed and had to be undone), `invalid_results` (times a robot was forced to stay, i.e. was truly blocked this step).

7.5 Complexity

Per the traceability notes, one timestep costs $O(|A|(\Delta(G) + F + \log|A|))$, where $|A|$ = robot count, $\Delta(G)$ = max graph degree (§1.1), F = per-candidate scoring cost (kept $O(1)$ via the occupancy index, spatial hash, and precomputed distance maps — no per-call BFS), and $\log|A|$ for maintaining priority order. PIBT's theoretical progress guarantee (that some robot always has a legal move) assumes every edge lies on a simple cycle (§1.3); one-way aisle constraints remove that freedom locally, which is why deadlock recovery (§10) exists as a safety net rather than a rare fallback.

8. Routing (`routing.py`)

In one sentence. How a robot works out a path to its next waypoint, and when that path is recomputed.

By default, `Router.route` is just `graph.shortest_route` (§1.2) — plain BFS. When `direction_aware_routing` is enabled *and* `direction_control == "aisle"`, it instead runs a **weighted A*** using the BFS distance map as an admissible, consistent heuristic:

```

edge_penalty(u, v):
    aisle ← aisle_of(v)
    if aisle is None or aisle.state = OPEN:      return 0
    if aisle.state = DRAINING:                  return route_direction_penalty (default 6.0)
    if traversal_direction(u,v) = aisle.current_direction: return 0
    else:                                       return route_direction_penalty

edge_cost(u, v) = 1 + edge_penalty(u, v)

```

The rationale (stated in the module docstring): PIBT's own layer only ever looks one step ahead, so directional information has to be baked into the *route itself*, or a robot will just sit at an aisle entrance repeatedly proposing (and being rejected for) the same illegal move until the aisle flips.

9. Task assignment (assignment.py)

In one sentence. Which robot gets which job. A greedy match over a cost that knows about distance, congestion, one-way aisles and how long the job has queued.

9.1 Directional delay

```

T_direction(source, goal) = Σ_{distinct aisles a on shortest_route(source, goal)} delay(a)

delay(a) = 0
           = occupancy(a) + 1
           = minimum_lock_time·0.5 + occupancy(a)
           = 0
           if a.state = OPEN
           if a.state = DRAINING
           if traversal direction ≠ a.current_direction (must wait for drain + flip)
           otherwise (direction already matches)

```

Only computed when `direction_control == "aisle"` ; each aisle counted once even if the route touches it multiple times.

9.2 Blocking estimate

```

B(robot, task) = (# bottleneck vertices on shortest_route(robot, task.pickup)) / max(1, route length)

```

9.3 Full assignment cost $J(i, \tau)$

$$J(i, \tau) = a \cdot d(x_i, p) + b \cdot d(p, d) + g \cdot C + d \cdot W + e \cdot T_{\text{direction}} + z \cdot B$$

TERM	CODE	DEFAULT COEFFICIENT
$a \cdot d(x_i, p)$	robot → pickup distance	<code>assign_alpha_to_pickup = 1.0</code>
$b \cdot d(p, d)$	pickup → delivery distance	<code>assign_beta_pickup_to_delivery = 0.5</code>
$g \cdot C$	<code>route_congestion(x_i, p)</code> , only if <code>congestion_aware</code>	<code>assign_gamma_congestion = 2.0</code>
$d \cdot W$	<code>task.waiting_time(t)</code>	<code>assign_delta_waiting = -0.5</code>
$e \cdot T_{\text{direction}}$	§9.1, only if <code>congestion_aware</code> or <code>direction_control="aisle"</code>	<code>assign_eta_direction = 1.0</code>
$z \cdot B$	§9.2, only if <code>congestion_aware</code>	<code>assign_zeta_blocking = 1.0</code>

Note $d = -0.5$ is **negative**: a longer-waiting task *lowers* its own cost, so it is preferred over a fresher, possibly-nearer task — this is the assignment layer's own (separate, additive) fairness mechanism, distinct from the priority function's waiting term (§3). Returns `INF` if either $x_i \rightarrow p$ or $p \rightarrow d$ is unreachable, so an infeasible pairing is never chosen.

Worked example. Same run, timestep 18. One unassigned task: pickup (0,0), delivery (15,0), released at $t = 6$ so it has been waiting 12 steps. Two candidate robots, costed by `TaskAssigner.assignment_cost`:

robot		$a \cdot d(r, p)$	$b \cdot d(p, d)$	$g \cdot C$	$d \cdot W$	$e \cdot T_{\text{dir}}$	$z \cdot B$	$J(i, \tau)$
robot 16 (0,4)	4	7.5	2.4	-6	0	0	7.9	
robot 18 (0,8)	8	7.5	2.667	-6	0	0	12.17	

Robot 16 wins at $J = 7.9$. Two columns are worth pausing on. $d(\text{pickup}, \text{delivery})$ is identical for both robots, because it does not depend on which robot goes — it is in the cost only so that a queue of tasks is ordered by total work rather than by how near its pickup happens to be. And $d = -0.5$ is *negative*, so the waiting term is a discount: the longer a task has been queued the cheaper it looks, which is how ageing gets into a greedy match.

That discount is capped at `assign_waiting_cap = 60`. Uncapped, `waiting_time` grows without bound in a lifelong run and dwarfs both distance and congestion within about a hundred steps; the match degenerates into oldest-task-first, and a congestion-aware assignment can no longer show any effect at all — which is exactly what H4 reported before the cap existed (§9.3).

9.4 Greedy matching

`assign_tasks_greedily` first pre-filters the unassigned-task pool to the oldest `max(assignment_candidate_limit, #free_robots)` tasks (sorted by `(release_time, id)` — a cheap bound so cost isn't computed against an unbounded backlog), then repeatedly:

```
while free robots and candidate tasks remain:
    (robot*, task*) ← argmin_{(robot, task) feasible} J(robot, task, t)
    assign robot* ← task*; remove both from their pools
```

This is **greedy minimum-cost matching**, $O(R \cdot T)$ per round for R robots and T candidate tasks — not the Hungarian algorithm, so it is not guaranteed globally optimal, only locally greedy at each pairing.

10. Deadlock detection and recovery (`deadlock.py`)

In one sentence. How the system tells a genuine deadlock apart from ordinary queueing — which in dense lifelong traffic is most of what it sees — and the seven increasingly drastic things it tries once it is sure.

10.1 Three distinct stall signals

The codebase tracks three related but different counters on `Robot`, worth disambiguating precisely since they feed different formulas:

COUNTER	MEANING	RESET CONDITION
<code>waiting_time</code>	steps spent physically stationary while not yet at the waypoint	resets whenever the robot moves or reaches its waypoint (simulator, §11 step 15)
<code>blocked_time</code>	steps in a row the PIBT recursion returned <code>INVALID</code> for this robot	resets to 0 on any <code>VALID result (pibt.py::plan_step)</code>
<code>no_progress_steps</code>	steps in a row <code>D_w(x_i)</code> did not strictly decrease	reset by <code>update_progress</code> on any route-distance improvement

10.2 Progress signal

$$\Delta_i(t) = D_w(x_i(t-1)) - D_w(x_i(t))$$

`update_progress` treats a robot as "not stalled" (resets `no_progress_steps = 0`) whenever it's idle (`FREE / PARKED`), already at its waypoint, or `D_w = 0`; otherwise it increments `no_progress_steps` unless `made_progress = (previous  $\neq$  INF  $\wedge$  current  $\neq$  INF  $\wedge$  current < previous)`.

$$\text{is_blocked}(\text{robot}) = \text{no_progress_steps} \geq t_blocked \vee \text{blocked_time} \geq t_blocked \quad (t_blocked \text{ default } 10)$$

10.3 Repeated-configuration and dependency-graph detection

Repeated configuration (a robot cycling through the same short loop of positions): with `period = 4`,

$$\text{repeated_configuration}(\text{robot}) = (\text{history}[-\text{period}:] = \text{history}[-2 \cdot \text{period} : -\text{period}])$$

i.e. the robot's last `period` positions exactly equal the `period` positions before that — `X(t-k) = X(t)` for `k = period`, checked on the 16-entry `position_history` deque.

Wait-for dependency graph: an edge `i → j` exists iff robot `i`'s `waiting_for_robot` (set during the PIBT recursion's priority-inheritance step, §7.4) currently equals robot `j`. `find_cycles` runs an iterative white/grey/black-colored DFS to find every simple directed cycle — a cycle here is a genuine circular wait (`i` waits for `j`, ..., waits for `i`), the textbook signature of deadlock.

`detect_deadlocked_groups` combines two signals: (1) any cycle in the wait-for graph containing at least one currently-blocked robot, and (2) a BFS-connected-component clustering of the *remaining* blocked robots within `radius = max(2, local_congestion_radius)` graph hops of each other — this second pass catches deadlocks that don't show up as a clean wait-for cycle (e.g. several robots each individually stuck against a static obstacle in the same local area) without needing a global graph traversal. Group lifetimes are tracked by `frozenset` membership keys across timesteps to count `detected` vs. `recovered` vs. `unrecovered`, and accumulate `recovery_time_total` for the mean-recovery-time metric (§12).

10.4 Seven-level progressive recovery

`recover_from_deadlock` escalates **one level per timestep** a given group persists (not all seven per call — see rationale below), applying the next level's remedy each time the group is still detected:

- 1. Recompute routes** — `shortest_route` from scratch for every robot in the group (cheapest possible fix: maybe the route was just stale).
- 2. Recompute affected aisles** — force `lock_until = t` on the group's current/next aisles (allow an immediate re-decision instead of waiting out the lock); if an affected aisle is empty, reopen it to `OPEN`; if occupied and directional, force it to `DRAINING`.

3. **Release stale reservations** — drop every aisle reservation held by the group.
4. **Increase blocked priorities** — `robot.priority += blocked_weight * no_progress_steps`, a direct, escalating priority boost on top of the regular formula (§3).
5. **Allow temporary reverse movement** — `allow_reverse_until = t + t_blocked`, `ignore_direction_until = t + max(1, t_blocked // 2)` (a shorter grace period than the reverse-move allowance).
6. **Assign escape vertices** — route each group member to its nearest unoccupied, unclaimed vertex from `_escape_vertices()` (parking bays, then passing bays, falling back to any degree- ≥ 3 vertex if the map has neither), and set `state = RECOVERY` (which also grants `priority_emergency`, §3.1).
7. **Run the local fallback planner** — the last resort: drop *all* directional constraints for the group (`ignore_direction_until = allow_reverse_until = t + t_deadlock`, default 20), release reservations, and restore task-directed state for any robot that was mid-recovery.

If a group is still detected after level 7 is applied, level 7's remedy keeps being reapplied every subsequent timestep and the group is counted once (not repeatedly) in `unrecovered`.

Why one level per timestep, not all seven per call: progress from a remedy can only be *observed* on the next timestep (the simulator is synchronous), so running all seven in one call would apply six of them blind — "progressive" recovery in a synchronous loop necessarily means one escalation step per detection cycle.

11. The lifelong simulation loop (`simulator.py`)

In one sentence. The sixteen steps that run, in this order, on every timestep. If you read one section to understand the system's shape, read this one.

`Simulator.step()` executes, per its own numbered comments, tying every prior section together in execution order:

1. Rebuild the occupancy index (§5.1); clear the per-timestep congestion cache (§5.2).
2. Receive newly arrived tasks (§2.3), if `lifelong`.
3. Update task/robot state transitions (§9, `update_task_state`); record completions; release aisle reservations for robots that just delivered.
4. Assign tasks greedily (§9.4); park idle robots (`_park_idle_robots`, below).
5. Update waypoints (§9's `update_waypoint`), `route_distance_to_waypoint`, proximity mode (§4.1), `direction_weight / aisle_weight` (§4.2–4.3), `current_aisle`.
6. Compute routes (§8); find `next_aisle` (`_find_next_critical_aisle`: the first upcoming route vertex whose aisle differs from the current one); compute the preferred direction with hysteresis (§4.7).
7. Aisle queues, direction decisions (§6.3), reservation grants (§6.5) — only if aisle management is `enabled`.
8. Deadlock detection and recovery (§10) — using the *previous* step's wait-for graph, since `waiting_for_robot` is only set during that step's PIBT run.
9. Compute every robot's `priority` (§3), sort descending; grant aisle reservations in that priority order (§6.5) if aisle-managed.
10. Run the low-level planner (`planner.plan_step`) — PIBT (§7), or a baseline (§14) via `planner_factory`.

11. Validate the joint plan (§7.4 companion, `validate_plan`) if `validate_every_step` — raises `PlanningError` on any residual vertex or swap conflict, which should be structurally impossible given §7.4 but is checked as a safety net.
12. Synchronized execution (`execute_moves` , §7.4 companion): every robot's move is applied atomically in one pass.
13. Update `waiting_time` (increments if stationary and not at the waypoint, else resets), `current_aisle` , `route_distance_to_waypoint` , call `update_progress` (§10.2), release aisle reservations for any robot now outside every aisle.
14. Rebuild the occupancy index again (positions changed).
15. Log the timestep's metrics (`TimestepRecord`).
16. Append a `StepSnapshot` to history, if `record_history` .

`_park_idle_robots` / `_choose_parking_vertex` : idle, task-less robots move to the nearest unclaimed parking bay or passing bay; if none is free, the robot simply **holds its current position** rather than driving toward an intersection — occupying an intersection would throttle every route through it, and PIBT's own priority inheritance (§7.4) will displace an idle robot the moment a busier one actually needs that cell.

`_raise_recursion_limit` : sets Python's recursion limit to `max(current, 100 + 12·n_robots)` — headroom for PIBT's recursive priority-inheritance chains, which in the worst case can be as deep as the robot count.

`run(max_timesteps, until_tasks, progress)` loops until the horizon, stopping early if `until_tasks` completions are reached or (non-lifelong mode only) every robot has reached its static goal.

12. Evaluation metrics (`metrics.py`)

In one sentence. What gets measured at the end of a run, and how each number is defined.

12.1 Percentile

Standard linear-interpolated percentile:

```
percentile(values, q):
    sort values; pos = (n-1)·(q/100); lower = [pos]; frac = pos - lower
    return values[lower]·(1-frac) + values[lower+1]·frac
```

12.2 Jain's fairness index

$$J = (\sum_i T_i)^2 / (n \cdot \sum_i T_i^2)$$

computed on `per_robot_tasks` (each robot's `completed_tasks` count). $J \in (0, 1]$; $J = 1$ iff every robot completed exactly the same number of tasks (perfect equality); $J \rightarrow 1/n$ as inequality maximizes (one robot does everything). This is the standard fairness measure from networking (Jain, Chiu, Hawe 1984), applied here to task-completion counts instead of throughput shares.

12.3 Aggregate report fields

`throughput = completed_tasks / max(1, timesteps)`; `makespan = max(completion_time)` over completed tasks; `mean/median/p95/max_service_time` from each completed task's `service_time = completion_time - release_time`; `total/mean_travel_distance =  $\Sigma$  robot.travel_distance` (incremented once per actual move in `execute_moves`); `direction_switches_per_1000 = 1000 · direction_switches / max(1, timesteps)`; plus the PIBT counters from §7.4 (`pibt_recursive_calls`, `pibt_backtracks`, `pibt_invalid_results`) and the deadlock counters from §10.3 (`deadlocks_detected/recovered/unrecovered`, `mean_recovery_time = recovery_time_total / recovered`).

13. Statistics (`stats.py`)

In one sentence. How to tell a real difference between two configurations from noise, without assuming the measurements are normally distributed.

Pure Python (no `scipy/numpy`), used by `experiments.run_comparison_table` to turn a handful of per-seed means into a defensible confidence interval and significance test.

13.1 Percentile bootstrap confidence interval

```
bootstrap_ci(values, n_resamples=10000, alpha=0.05):
    repeat n_resamples times: draw n values from `values` *with replacement*, take their mean
    sort the n_resamples resample-means
    lo = resample_means[(alpha/2)·n_resamples]
    hi = resample_means[(1 - alpha/2)·n_resamples]
    return (mean(values), lo, hi)
```

This is the standard **percentile bootstrap** for a $(1-\alpha)$ confidence interval on the mean — no distributional assumption is made about `values` beyond that resampling with replacement approximates its sampling distribution. Degenerates to `(m, m, m)` for fewer than 2 values (no variance can be estimated from a single sample).

13.2 Permutation test

```
permutation_test(a, b, n_permutations=10000):
    observed = mean(a) - mean(b)
    threshold = |observed|
    pooled = a + b
    total_splits = C(len(pooled), len(a))
    if total_splits ≤ 20000:
        p = ( # of every possible way to split `pooled` into groups the size of a,b
              whose |mean-difference| ≥ threshold - 1e-12 ) / total_splits    [exact enumeration]
    else:
        p = ( # of n_permutations random shuffles of `pooled` meeting the same bound ) / n_permutation
    return (observed, p)
```

This is a standard **two-sided permutation test**: under the null hypothesis that `a` and `b` are draws from the same distribution, every relabeling of the pooled data into two groups of the original sizes is equally likely, so the p-value is the fraction of relabelings at least as extreme as what was actually observed. Below `C(n, |a|) ≤ 20000` splits, every split is enumerated exactly via `itertools.combinations` (deterministic, seed-independent); above that, `n_permutations`

random shuffles are sampled via a seeded `random.Random` (deterministic given the seed). Special case: `p = 1.0` if either group is empty or spans the whole pool (no valid split exists).

14. Baseline algorithms (`baselines/`)

In one sentence. The two published lifelong algorithms LDA-PIBT is compared against, and the space-time search both of them are built on.

Built to give LDA-PIBT something from the literature to be compared against, since every ablation-table comparison elsewhere in the codebase is this codebase against itself. Both baselines reuse the identical `assignment.TaskAssigner` — only the low-level movement/collision-avoidance layer differs, isolating that layer's contribution.

14.1 Time-expanded A* (`space_time_search.py`)

The shared primitive both baselines build on: **cooperative A*** over states `(vertex, t)` rather than plain `vertex`, since avoiding *other robots* requires reasoning about *when* a cell is occupied, not just whether it's passable.

```
g(vertex, t) = path cost so far;  f = g + h(vertex)    where h = precomputed BFS distance map to goal (§1.2)
```

`h` is admissible and consistent on this unweighted grid graph (it's the true unweighted shortest-path distance, and every move costs exactly 1), so the A* search is optimal. Each state transition is a move to any neighbor, or waiting in place, each costing `+1`, filtered through `reservations.is_free` (checks both a vertex hold and — critically — a *reverse-edge* hold, to catch swap conflicts the same way `piibt.py`'s `creates_swap_conflict` does, §7.3.2).

Two modes:

- `require_goal=True` (Token Passing, §14.2): only returns a path that actually reaches the goal *and* can hold there for the rest of the search horizon (`goal_is_clear`) — the standard cooperative-A* "goal condition", ensuring a robot doesn't stop somewhere another robot will need to pass through later.
- `require_goal=False` (RHCR, §14.3): the horizon is a short rolling window decoupled from actual goal distance, so reaching the goal within one window is the exception. Instead returns the **best-effort** path: if goal is never reached by `t = start_time + horizon`, return the path to whichever boundary state has the smallest `h` value (closest to goal), ties broken by A*'s pop order.

`prioritized_plan`: plans robots **one at a time**, committing each resulting path into the shared reservation table before planning the next — classic sequential/prioritized MAPF decomposition (no joint-optimality guarantee, but always fast and always produces *some* legal plan). Every robot's *current* cell is pre-held for one extra timestep so a later-in-order, not-yet-planned robot can't path straight through a robot that hasn't moved yet.

`resolve_residual_conflicts`: a post-hoc safety net run after both baselines finish planning — repeatedly finds any residual vertex/swap conflict among the committed `next_position`s and downgrades the lower-priority robot (by index in the simulator's own priority order) to hold in place, iterating until stable (bounded by `n+1` iterations, since each iteration removes at least one conflict).

14.2 Token Passing (Ma, Kumar, Koenig & Ayanian, 2017)

Each step: any robot whose committed path is stale (start/goal mismatch, or exhausted) gets replanned via `prioritized_plan` with `require_goal=True` against a table pre-seeded by holding every *still-valid* path of settled robots. A robot whose search fails simply **waits** in place (`path_not_found` counter) — Token Passing, as published, has **no priority-inheritance or backtracking mechanism**: a blocked robot cannot ask the robot in its way to move. This is the structural reason (not a bug in this implementation) for the corridor-gridlock failure mode the README documents empirically: once a queue forms in a single-file corridor with no mechanism to displace anyone, it never resolves.

14.3 RHCR — Rolling-Horizon Collision Resolution (Li, Tinka, Kiesel, Durham, Kumar & Koenig, 2021)

Every `replan_period` steps (default 5), all robots are jointly replanned over a `window`-length horizon (default 10) via **windowed Conflict-Based Search** (Sharon et al.'s CBS, restricted to the window):

```
_CBSNode = (constraints: robot → set of (vertex,t) / (u,v,t) forbidden states, paths: robot → path)

root: each robot's own unconstrained space_time_astar path (require_goal=False)
cost(node) = Σ_robots len(path)                                (sum-of-costs)
best-first search over nodes, ordered by cost:
  pop lowest-cost node
  conflict ← first_conflict(node.paths)                        (scan timestep-by-timestep within the window
                                                                for a shared vertex, or a reversed-edge swap
  if no conflict: return node.paths                            # done
  branch into two children, each adding one of the two constraints from `_branch(conflict)`
  for each child: replan only the newly-constrained robot's path; push if feasible
```

`_branch` on a vertex conflict `(i, j, v, t)` produces the two children `{i forbidden at (v,t)}` and `{j forbidden at (v,t)}`; on an edge/swap conflict `(i, j, (u,v), t)`, `{i forbidden the edge u→v at t}` and `{j forbidden the edge v→u at t}`. This is exactly Sharon et al.'s conflict-splitting rule. If node expansions exceed `node_expansion_cap` (500) before a conflict-free node is found, the search **falls back to `prioritized_plan`** for that window (`cbs_fallbacks` counter) — standard, documented RHCR practice for bounding CBS's worst-case exponential blowup, not a shortcut unique to this implementation.

Between periodic full replans, `_repair_stale_paths` gives any robot with a newly-assigned task or an exhausted path an immediate single-agent repair via `prioritized_plan`, rather than making it wait up to `replan_period` steps for the next window.

15. Parameter reference (`config.Params`)

In one sentence. Every tunable in one table: what it does, what it defaults to, and which ablation variant switches it.

Every tunable constant, grouped by the section above that uses it.

Candidate scoring (§4): `alpha_progress=10.0`, `beta_strong=3.0`, `beta_weak=1.0` (*defined but currently unused by any formula above — `compute_direction_weight` only reads `beta_strong`*), `gamma_strong=2.0`, `gamma_weak=0.5`, `lambda_turn=0.5`, `lambda_reverse=2.0`, `mu_congestion=1.0`, `nu_wait=0.2`, `xi_bottleneck=1.0`, `zeta_counterflow=8.0`, `zeta_reservation=8.0`, `progress_normalization="step"`, `hard_direction_constraints=False`.

Congestion (§5): `omega_local=1.0`, `omega_aisle=1.0`, `omega_downstream=1.0`,
`local_congestion_radius=3`, `downstream_horizon=5`, `congestion_normalisation=True`.

Proximity (§4.1): `r_near=2`, `r_far=8`.

Aisle management (§6): `minimum_aisle_lock_time=20`, `maximum_aisle_lock_time=40`,
`direction_switch_threshold=5.0`, `aisle_capacity=10`, `aisle_capacity_model="drain"` (`"length"` =
spec 9.2, `"throughput"` = exit-gated), `aisle_capacity_ratio=1.0`, `reservation_ttl=15`,
`max_drain_time=30`, `directional_aisle_min_length=4`, `direction_aware_routing=False`,
`route_direction_penalty=6.0`.

Three extensions beyond the spec, all off by default and all kept because their measured effect is worth being able to reproduce: `parity_bias=0.0` (a per-aisle-parity bias meant to make neighbouring aisles prefer opposite directions — it does not fix the parallel-corridor lockstep it targets), `coordinate_aisle_directions=False` (§6.4.1 — a null result), and `demand_spread=False` (§6.3 — net negative).

Directional demand (§6.2): `w_urgency=1.0`, `w_waiting=0.5`, `w_proximity=2.0`, `w_route_length=0.05`,
`w_congestion=0.5`.

Priority (§3): `priority_emergency=400.0`, `priority_loaded=300.0`, `priority_pickup=200.0`,
`priority_repositioning=100.0`, `priority_free=0.0`, `priority_inside_aisle=50.0`,
`waiting_weight=5.0`, `blocked_weight=10.0`, `urgency_weight=1.0`.

Deadlock (§10): `t_blocked=10`, `t_deadlock=20`, `require_deadlock_corroboration=True`,
`config_history_length=8` (*reserved field; not read by any formula above*).

Task assignment (§9): `assign_alpha_to_pickup=1.0`, `assign_beta_pickup_to_delivery=0.5`,
`assign_gamma_congestion=12.0`, `assign_delta_waiting=-0.5`, `assign_waiting_cap=60.0`,
`assign_eta_direction=1.0`, `assign_zeta_blocking=1.0`, `assignment_candidate_limit=32`.

`assign_gamma_congestion` was 2.0 against distances of 10-40, so the whole congestion term was worth about four cost units and could not influence a match; `assign_waiting_cap` bounds a term that otherwise grows without limit in a lifelong run and degenerates the match to oldest-task-first.

Ablation switches (§16): `lifelong=True`, `direction_control="aisle"` (`"none"` | `"robot"` | `"aisle"`),
`hysteresis=True`, `congestion_scoring=True`, `congestion_assignment=True`, `reservations=True`,
`recovery=True`, `turning_cost=True`.

`congestion_aware` survives as a read/write alias that sets and reads both congestion flags, so saved configs and CLI overrides keep working. It was one flag switching three mechanisms — the movement penalty, the assignment congestion term and the assignment blocking term — which made H4, a claim about matching alone, unmeasurable.

Simulation: `max_timesteps=500`, `seed=0`, `park_when_idle=True`, `validate_every_step=True`.

Baseline planners (§14): `baseline_window=10` (RHCR only), `baseline_replan_period=5` (RHCR only).

15.1 The ablation ladder and factorial designs

`config.ABLATIONS` defines six cumulative named presets, each turning on one more mechanism than the last:

`pibt_baseline` (everything off, non-lifelong) → `lifelong_pibt` → `directional_pibt`
(`+direction_control=robot`, `+turning_cost`) → `hysteresis_pibt` (`+hysteresis`) →
`aisle_managed_pibt` (`+direction_control=aisle`, `+reservations`) → `full_lda_pibt`
(`+congestion_aware`, `+recovery`).

Three of these six steps flip **two** flags at once, so a measured throughput delta on those steps can't be attributed to either flag alone. `FACTORIAL_DESIGNS` resolves this with a standard **2×2 factorial decomposition** for each such step, run via `experiments.run_factorial_table`:

```
main_effect_a      = value(factor_a alone) - value(base)
main_effect_b      = value(factor_b alone) - value(base)
additive_prediction = value(base) + main_effect_a + main_effect_b
observed_both_effect = value(both) - value(base)
interaction         = observed_both_effect - main_effect_a - main_effect_b
```

A large `interaction` term means the two flags are strongly non-additive — e.g. the finding that on `warehouse_medium` aisle-level direction alone *raises* throughput (0.337 → 0.405) while entry admission alone lowers it (→ 0.262), and the two together land at 0.217, well below what either main effect predicts.

Two cautions the earlier version of this table did not carry, both learned the hard way:

- **A factorial is only readable if all four corners are distinct runs.** The `aisle_direction_vs_reservations` design was unreadable for exactly one reason: `reservations_only` sets `direction_control="robot"`, and the reservation layer was gated on `direction_control == "aisle"`, so factor B was a bit-identical copy of the base and its main effect was *necessarily* zero. "Exactly zero effect" was a property of the wiring, not of reservations. Before reading any decomposition, check that the isolated cell actually differs from the base.
- **A flag that switches more than one mechanism cannot be a factor.** `congestion_aware` moved the movement score, the assignment cost and the assignment blocking term together — the same confound the factorials exist to remove, one level down. `congestion_scoring_vs_assignment` splits it.

`config.PAIRED_DESIGNS` covers the cases where a 2×2 is impossible because the second factor is undefined when the first is off — there is no "hard constraint" cell without aisle direction, and no "corroboration" cell without recovery. `experiments.run_paired_table` reports those as single-factor comparisons with the same bootstrap CI and permutation test.

16. Glossary of symbols

In one sentence. Every symbol used above, in one table, with the module that owns it.

SYMBOL	MEANING	MODULE
$x_i(t)$	position of robot i at time t	robot.py
$D_g(v)$	BFS graph distance from v to goal g	graph.py
$\Delta(G)$	max vertex degree in the graph	graph.py
r_i	route distance from robot i to its waypoint	scoring.py
R_{near}, R_{far}	proximity-mode thresholds (2, 8)	config.py
β_i	smooth direction-preference weight	scoring.py
γ_i	aisle-continuity weight	scoring.py
$\alpha, \lambda_{turn}, \mu, \nu, \xi$	candidate-score weights (progress, turn, congestion, wait, bottleneck)	scoring.py
$\eta_{reverse}$ ($\lambda_{reverse}$ in code)	180°-reversal turn-cost multiplier	scoring.py
$S_i(v)$	full candidate score for robot i moving to v	scoring.py
$C_i(v)$	congestion estimate at candidate v	congestion.py
$\omega_{local}, \omega_{aisle}, \omega_{downstream}$	congestion mixture weights	config.py
S_a^+, S_a^-	aggregate forward/reverse directional demand for aisle a	aisle_manager.py
τ_{switch}	direction-switch hysteresis threshold	aisle_manager.py
$p_i(t)$	robot i 's scheduling priority at time t	priority.py
P_{class}	task-class base priority	priority.py
k_w, k_b, k_u, k_e	priority weights on waiting/blocked/urgency/in-aisle	priority.py (code: <code>waiting_weight</code> , <code>blocked_weight</code> , <code>urgency_weight</code> , <code>priority_inside_aisle</code>)
ϵ_i	per-robot tie-breaker ($1e-4 \cdot id$)	robot.py
U_i	task urgency	task.py
W_i	waiting-time term (usage varies: robot <code>waiting_time</code> in <code>priority.py</code> ; a stall-aware max in <code>aisle_manager.py</code>)	priority.py, aisle_manager.py
$J(i, \tau)$	task-assignment cost	assignment.py
$T_{direction}$	expected aisle-direction delay along a route	assignment.py
$B(i, \tau)$	route bottleneck-blocking estimate	assignment.py
$\Delta_i(t)$	one-step route-distance progress	deadlock.py
J (Jain's index)	task-completion fairness across robots, $\in (0, 1]$	metrics.py
$C_i = N(x_i) \cup \{x_i\}$	PIBT candidate set	pibt.py
INF	<code>float("inf")</code> , the unreachable-distance sentinel	types.py

Cross-reference

For the mapping from each formula above back to the (external, not included in this repo) numbered spec sections, see `docs/implementation-notes.md` . For the empirical validation of these formulas — which ablations actually help, which hypotheses held up under de-confounded 2×2 testing (§15.1), and how LDA-PIBT compares against the two baselines of §14 — see the "Results" section of `README.md` .